SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	S.N.V.S Karthik	**Reg. No.:**	192424186
Section / Batch:	B Tech AIDS 2024	**Date:**	

Code of Conduct

I SS.N.V.S Karthik 192424186 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____karthik_____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

CODE

```
words = [
    "treatment",
    "treatable",
    "retreatment",
    "treated",
    "untreated"
]
def analyze(word):
    prefix = ""
    suffixes = []
    root = word
    if root.startswith("re"):
        prefix = "re"
        root = root[2:]
    elif root.startswith("un"):
        prefix = "un"
        root = root[2:]
    if root.endswith("ment"):
        suffixes.append(("ment", "Derivational"))
        root = root[:-4]
    elif root.endswith("able"):
        suffixes.append(("able", "Derivational"))
        root = root[:-4]
    if root.endswith("ed"):
        suffixes.append(("ed", "Inflectional"))
        root = root[:-2]
    return prefix, root, suffixes
for word in words:
    prefix, root, suffixes = analyze(word)
    print("\nWord:", word)
    print("Prefix:", prefix if prefix else "None")
    print("Root:", root)
    if suffixes:
        for suffix, typ in suffixes:
            print("Suffix:", suffix, "->", typ)
    else:
        print("Suffix: None")
```

```
...
    Word: treatment
    Prefix: None
    Root: treat
    Suffix: ment -> Derivational

    Word: treatable
    Prefix: None
    Root: treat
    Suffix: able -> Derivational

    Word: retreatment
    Prefix: re
    Root: treat
    Suffix: ment -> Derivational

    Word: treated
    Prefix: None
    Root: treat
    Suffix: ed -> Inflectional

    Word: untreated
    Prefix: un
    Root: treat
    Suffix: ed -> Inflectional
```

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

CODE

```
words = [
    "replayed",
    "unhappier",
    "disconnected",
    "players",
    "restarting",
    "unreadable"
]
prefixes = ["re", "un", "dis"]
suffixes = ["able", "ing", "ed", "er", "s"]
def parser(word):
    original = word
    prefix = ""
    suffix_list = []
    root = word
    for p in prefixes:
        if root.startswith(p):
            prefix = p
            root = root[len(p):]
            break
```

```
    changed = True
    while changed:
        changed = False
        for s in suffixes:
            if root.endswith(s) and len(root) > len(s):
                suffix_list.insert(0, s)
                root = root[:-len(s)]
                changed = True
                break
    return prefix, root, suffix_list
expected = {
    "replayed": ("re", "play", ["ed"]),
    "unhappier": ("un", "happy", ["er"]),
    "disconnected": ("dis", "connect", ["ed"]),
    "players": ("", "play", ["er", "s"]),
    "restarting": ("re", "start", ["ing"]),
    "unreadable": ("un", "read", ["able"])
}
correct = 0
for word in words:
    result = parser(word)
    print("\nWord:", word)
    print("Parsed:", result)
    print("Expected:", expected[word])
    if result == expected[word]:
        correct += 1
accuracy = correct / len(words) * 100
print("\nAccuracy:", accuracy, "%")
```

```
Word: replayed
Parsed: ('re', 'play', ['ed'])
Expected: ('re', 'play', ['ed'])

Word: unhappier
Parsed: ('un', 'happi', ['er'])
Expected: ('un', 'happy', ['er'])

Word: disconnected
Parsed: ('dis', 'connect', ['ed'])
Expected: ('dis', 'connect', ['ed'])

Word: players
Parsed: ('', 'play', ['er', 's'])
Expected: ('', 'play', ['er', 's'])

Word: restarting
Parsed: ('re', 'start', ['ing'])
Expected: ('re', 'start', ['ing'])

Word: unreadable
Parsed: ('un', 'read', ['able'])
Expected: ('un', 'read', ['able'])

Accuracy: 83.33333333333334 %
```

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.

3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
    - Original text
    - Tokens
    - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.
7. Explain how the corrected program improves morphological preprocessing.

CODE

```
import re
from nltk.stem import PorterStemmer
ps = PorterStemmer()
documents = [
    "Researchers studied new treatments for patients.",
    "Scientists are studying medical discoveries.",
    "The government announced new economic policies.",
    "Companies are developing advanced technologies.",
    "Doctors treated patients with new medicines."
]
def process(text):
    tokens = re.findall(r"[A-Za-z]+", text.lower())
    stems = [ps.stem(word) for word in tokens]
    return tokens, stems
for text in documents:
    tokens, stems = process(text)
    print("\nOriginal Text:", text)
    print("Tokens:", tokens)
    print("Stemmed Tokens:", stems)
words = [
    "studies",
    "studied",
    "studying",
    "cats",
    "dogs",
    "boxes",
    "houses",
    "running",
    "walked",
    "played",
    "relational",
    "organization",
    "organizational",
    "happiness",
    "hopeful",
    "kindness",
    "readable",
    "connectivity",
    "adjustment",
    "replacement"
]
print("\n20 Stemming Cases:")
for word in words:
    print(word, "->", ps.stem(word))
```

```
Original Text: Researchers studied new treatments for patients.
Tokens: ['researchers', 'studied', 'new', 'treatments', 'for', 'patients']
Stemmed Tokens: ['research', 'studi', 'new', 'treatment', 'for', 'patient']

Original Text: Scientists are studying medical discoveries.
Tokens: ['scientists', 'are', 'studying', 'medical', 'discoveries']
Stemmed Tokens: ['scientist', 'are', 'studi', 'medic', 'discoveri']

Original Text: The government announced new economic policies.
Tokens: ['the', 'government', 'announced', 'new', 'economic', 'policies']
Stemmed Tokens: ['the', 'govern', 'announc', 'new', 'econom', 'polici']

Original Text: Companies are developing advanced technologies.
Tokens: ['companies', 'are', 'developing', 'advanced', 'technologies']
Stemmed Tokens: ['compani', 'are', 'develop', 'advanc', 'technolog']

Original Text: Doctors treated patients with new medicines.
Tokens: ['doctors', 'treated', 'patients', 'with', 'new', 'medicines']
Stemmed Tokens: ['doctor', 'treat', 'patient', 'with', 'new', 'medicin']

20 Stemming Cases:
studies -> studi
studied -> studi
studying -> studi
cats -> cat
dogs -> dog
boxes -> box
houses -> hous
running -> run
walked -> walk
```

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
if word.endswith("s"):
    return word[:-2], "Plural Noun"
else:
    return word, "Singular"

words = [
  "cars",
  "boxes",
  "cities",
  "children",
  "books"
]

for w in words:
  print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
    - Regular plurals
    - Words ending in -es
    - Words ending in -ies
    - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

CODE

```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "people": "person",
        "mice": "mouse",
        "feet": "foot",
        "teeth": "tooth"
    }
    if word in irregular:
        return irregular[word], "Plural Noun"
    if word.endswith("ies"):
        return word[:-3] + "y", "Plural Noun"
    if word.endswith("es"):
        return word[:-2], "Plural Noun"
    if word.endswith("s"):
        return word[:-1], "Plural Noun"
    return word, "Singular"
words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]
for word in words:
    print(word, "->", parser(word))
import nltk
from nltk.corpus import wordnet as wn
try:
    wn.synsets("child")
except LookupError:
    nltk.download("wordnet")
print("\nWordNet Validation")
for word in words:
    root, typ = parser(word)
    synsets = wn.synsets(root, pos=wn.NOUN)
    print(
        word,
        "->",
        root,
        "->",
        typ,
        "WordNet meanings:",
        len(synsets)
    )
```

output actions

```
cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Plural Noun')
books -> ('book', 'Plural Noun')
[nltk_data] Downloading package wordnet to /root/nltk_data...

WordNet Validation
cars -> car -> Plural Noun WordNet meanings: 5
boxes -> box -> Plural Noun WordNet meanings: 10
cities -> city -> Plural Noun WordNet meanings: 3
children -> child -> Plural Noun WordNet meanings: 4
books -> book -> Plural Noun WordNet meanings: 11
```

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy before correction
    - Classification accuracy after correction
    - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

CODE

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
stemmer = PorterStemmer()
documents = [
   "connected connection connecting connectivity",
   "studies studied studying study",
   "organize organized organizer organization"
]
vectorizer1 = CountVectorizer()
X1 = vectorizer1.fit_transform(documents)
vocab_before = vectorizer1.get_feature_names_out()
print("Vocabulary Before:")
print(vocab_before)
print("Vocabulary Size Before:",
     len(vocab_before))
normalized_documents = []
for document in documents:
   words = document.lower().split()
   stems = []
   for word in words:
      stems.append(stemmer.stem(word))
   normalized_documents.append(" ".join(stems))
print("\nNormalized Documents:")
for document in normalized_documents:
   print(document)
vectorizer2 = CountVectorizer()
X2 = vectorizer2.fit_transform(normalized_documents)
```

```
vocab_after = vectorizer2.get_feature_names_out()
print("\nVocabulary After:")
print(vocab_after)
print("Vocabulary Size After:",
      len(vocab_after))
print("\nOriginal Feature Matrix:")
print(X1.toarray())
print("\nCorrected Feature Matrix:")
print(X2.toarray())
examples = [
    "connected",
    "connection",
    "connecting",
    "connectivity",
    "studies",
    "studied",
    "studying",
    "study",
    "organize",
    "organization"
]
print("\nMorphological Normalization")
for word in examples:
    print(word, "->", stemmer.stem(word))
```

```
Vocabulary Before:
['connected' 'connecting' 'connection' 'connectivity' 'organization'
 'organize' 'organized' 'organizer' 'studied' 'studies' 'study' 'studying']
Vocabulary Size Before: 12

Normalized Documents:
connect connect connect connect
studi studi studi studi
organ organ organ organ

Vocabulary After:
['connect' 'organ' 'studi']
Vocabulary Size After: 3

Original Feature Matrix:
[[1 1 1 1 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 1 1 1 1]
 [0 0 0 0 1 1 1 1 0 0 0 0]]

Corrected Feature Matrix:
[[4 0 0]
 [0 0 4]
 [0 4 0]]

Morphological Normalization
connected -> connect
connection -> connect
connecting -> connect
connectivity -> connect
studies -> studi
studied -> studi
studying -> studi
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Dr. T Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____